Software Development in an AI World

A Reflection on My Product Internship at Samba TV

Lauren Lambrecht

Claremont McKenna College

Fall 2026

Table of Contents

I. Introduction 3

II. About the Internship 5

III. Building a Product: My Internship Experience 8

IV. AI in My Development Workflow 12

V. How AI Is Changing Software Development 16

VI. The Developer's Role in an AI World 20

VII. Limitations and Risks of AI-Assisted Development 23

VIII. What My Internship Taught Me About the Future of Software 26

IX. Conclusion 28

X. References 30

I. Introduction

On my first day at Samba TV, someone on my team said the word “Databricks” and I nodded like I knew what it was. I did not know what it was. I was a rising senior studying International Relations and History, not computer science, and the sum total of my coding background was one basic Python course and a vague memory of what a for-loop does. Ten weeks later, I had built four working app prototypes, each one pulling real viewership data, each one tested with real users, and each one written up into a set of findings that my team actually used to decide what to build next. I did not become a software engineer over the course of one summer. What happened instead is a little stranger, and it is the subject of this paper: I got to build real software, at a real company, without ever really learning to code in the traditional sense, because an AI tool did most of the actual coding, and my job became something else entirely.

That something else is what this paper is really about. The easy version of the “AI is changing software” story is that AI writes code now, so programmers are becoming obsolete. I do not think that is true, and my internship is not really evidence for it either way, since I was never a programmer to begin with. What I saw instead was more specific: the parts of building software that used to require years of technical training (i.e. writing a working query, wiring up a backend, tracking down a bug in a language you barely know) became fast and approachable, almost incidental. And the parts that used to be secondary to “real” engineering (deciding what to build, figuring out whether it actually worked for a human being, and knowing when to kill a feature you just spent two weeks building) became the entire job.

One of the first users I interviewed, a woman named Yulia, told me she felt “a little bit lied to” every time Netflix recommended her its own movies instead of something she would

actually enjoy. She was not talking about our prototype. She was talking about the fact that she had never fully trusted a recommendation engine with a financial stake in what she watched.

This paper walks through what that looked like in practice— the product I helped build over four sprints at Samba TV, the specific ways I leaned on AI to do it, and what I think that experience says about where the job of "software developer" is actually headed. My answer, in short, is that AI is not replacing the people who build software. It is changing the altitude they build from.

II. About the Internship

I spent this past summer, June 15 through August 14, as a Product Intern at Samba TV, working on the team rebuilding the company's consumer mobile app. Samba TV is not a household name the way Netflix or Hulu are, but there is a good chance it has quietly known what was playing on your television at some point. The company built its business on automatic content recognition technology embedded at the chipset level across more than two dozen TV brands. In plain terms, that means Samba does not need anyone to download an app or log into anything for its data to exist— it already knows, at the hardware level, what is playing on a huge share of the smart TVs in the country. That is a genuinely different starting position than almost any competitor, whether that is a single streaming platform that only sees what happens inside its own app, or an app like Trakt that needs a person to opt into tracking through a separate piece of software. Samba's data exists whether or not anyone ever opens the Samba app at all. The app is just a layer on top of something that was already there.

Beyond the automatic content recognition piece, four things about Samba's position stood out to me over the summer, and they ended up shaping almost every decision I made about what to prototype. The first is cross-platform reach: because the tracking happens at the chipset level, Samba can see viewing behavior across more than two dozen TV brands rather than being locked into one operating system, which is exactly what let a feature like a truly cross-platform recommendation engine make sense in the first place. The second is that hardware advantage directly. Essentially, Samba is already embedded at the point where someone sets up a new television, so it does not depend on marketing to get discovered the way a standalone app does. The third is the depth of the underlying data infrastructure: the information exists whether or not the consumer app gets adopted at all, which meant the app I was prototyping was always a layer

on top of something durable, not the thing keeping the lights on. And the fourth is that all of it is opt-in and disclosed to the user, a genuinely different starting point from a company whose business model depends on a person never reading the terms of service closely. I did not fully appreciate how much that last point mattered until I started running interviews and kept hearing testers describe wanting to know, specifically, what a company was doing with their data before they would trust anything else about it.

My own coursework going into the summer was closer to this last point than to anything technical— a mix of international relations, history, and a few classes touching business and political economy, all of which turned out to be more useful for understanding a company's market position than I expected, even if none of it prepared me to open Databricks for the first time.

That hardware advantage is also what made my internship possible in the first place. Samba's actual consumer-facing mobile app had fallen behind. It did the basics, but had not been meaningfully rebuilt in a while, and the team wanted to know what a second version should include before committing real engineering resources to it. That timing mattered for a reason outside the company, too: Trakt, the tracking app most serious TV watchers already used, doubled its subscription price and started paywalling core features in 2025, and a visibly frustrated user base started looking for alternatives (AlternativeTo, 2025). Samba had the underlying data advantage to become that alternative. What it did not have yet was a product.

That is where I came in. My job for the summer was to figure out what a "v2" of the Samba mobile app should actually be, and to do it in a way that did not just guess— every idea had to become a real, functioning prototype that real people could test before it went anywhere near an engineering roadmap. The internship was structured around four two-week sprints, each

built around a different feature theme: discovery-era recommendations and rewards first, then engagement features like tracking and a social layer, then more experimental interactive features like live voting and shoppable content, and finally a foundation sprint focused on onboarding and privacy. Each sprint followed the same rhythm: build a working prototype in the first few days, run five to eight one-on-one interviews with prospective users to test it, and then write up what we learned into a ranked set of recommendations before moving into the next sprint.

I want to be upfront about something that shapes everything else in this paper: I was not hired because I could code. I was hired, as best I can tell, because I could talk to people, notice patterns in what they said, and hold a coherent point of view about what a product should do. The coding part of the job is something I picked up almost entirely on the job, in real time, with an AI coding tool doing most of the heavy lifting. That is not a small detail. It is the reason I can even write the rest of this paper.

III. Building a Product: My Internship Experience

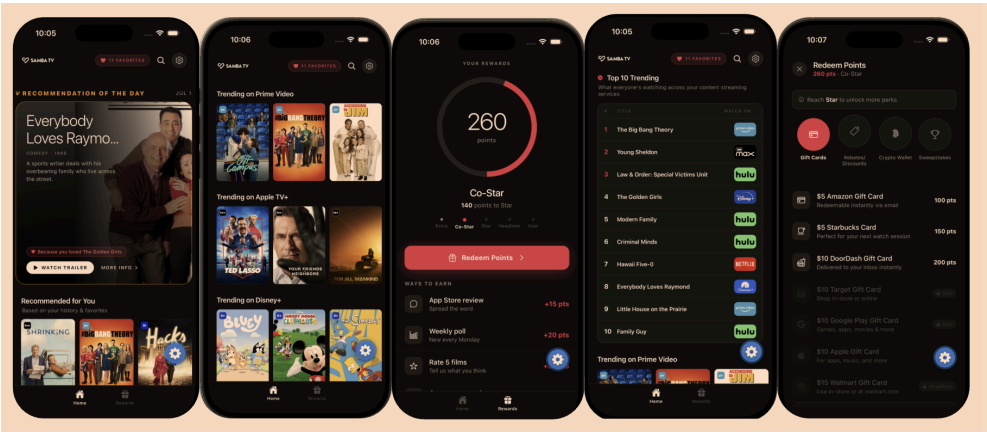
Each sprint at Samba followed the same basic arc compressed into about ten working days: idea, design, build, test, debug, review. That pace sounds aggressive on paper, and it was, but it is also the only way I can explain how someone with as little technical background as I had could ship four real prototypes in a single summer. I want to walk through what actually got built, sprint by sprint, because the specific features matter less than what building them taught me about the process itself.

The testing side of each sprint mattered just as much as the building side, and it is worth describing what an actual session looked like, because it shaped how I thought about every prototype after the first one. Each interview ran forty-five minutes to an hour, and followed the same three-part structure regardless of sprint: first a short profile assessment of how that person actually watches television today, how many services they subscribe to, whether they watch alone or with a partner or kids, and how they currently decide what to watch next; then hands-on testing, where I would hand them the prototype itself and watch, mostly in silence, as they tried to use it without me explaining anything; and finally a closing conversation about whether they would actually come back to something like this on a normal Tuesday night. Across the summer our participant pool ranged from people managing two subscriptions to people managing seven or eight, solo apartments to shared family accounts, and that spread mattered, because a feature that thrilled a heavy streamer with five services open at once sometimes fell completely flat for someone who mostly just wanted to know what was new on the one service they already paid for. Watching someone sit in silence trying to figure out what a button did, without me jumping in to explain it, was uncomfortable in a way that reading a bug report never is, and it is probably the single habit from this internship I expect to carry the furthest.

Sprint 1: Recommendations and Rewards

The first sprint, in mid-June, tested Samba's most obvious pitch: a cross-platform recommendation engine that was not secretly trying to sell you anything, paired with a rewards system that gave people a reason to share their viewing data. I built a working prototype with a home screen of recommendation carousels pulled from multiple streaming services at once, plus a rewards screen with a point system and redeemable gift cards. The "pulled from multiple services at once" part was the hard piece for me technically. This meant I had to go into Databricks, Samba's data platform, and write queries that returned real trending and viewership data instead of me hand-typing a list of shows into the prototype. I had touched basic Python before that summer. I had never touched Databricks. Getting a query to run, understanding why it returned the wrong thing, and turning that raw data into something a recommendation carousel could display took me longer than building the screens themselves.

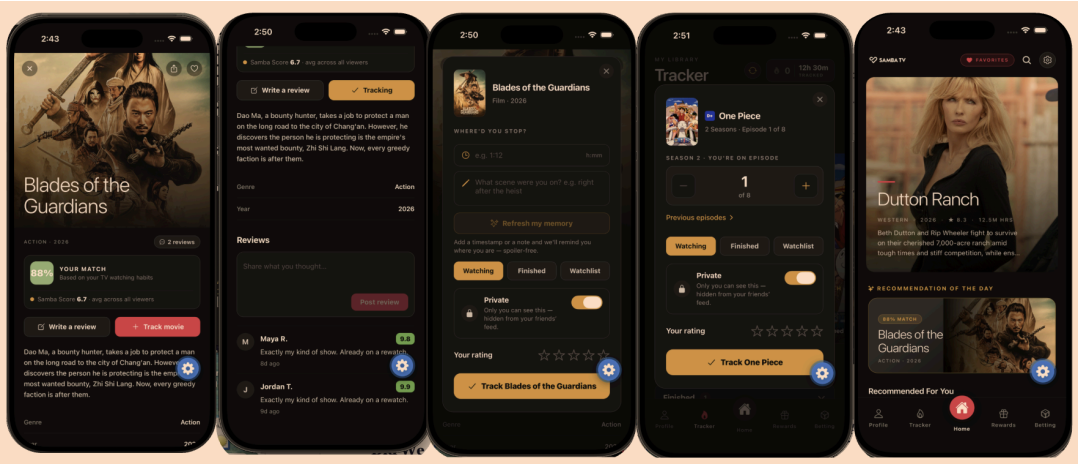
We tested that prototype with six people, and the sessions taught me something no amount of building alone would have: the feature I was proudest of, the reward tier system, was completely unintelligible to every single person who touched it. Watching that happen live, across six different calls, is a different kind of feedback than a code review. Nothing was broken. It ran exactly as it was built. And yet, it did not hit consumers as expected. This was our first of many lessons.



Sprint 2: Tracking and Social

The second sprint moved into engagement: a redesigned recommendations page pairing a personal match score with an aggregate score across all users, a tracker for shows and movies, a scene-recall tool we called “Refresh My Memory,” a viewing-habits screen, and a friend’s feed. This was the sprint where the technical lift grew. Throughout, I was syncing tracked-show data, building out the Firebase backend that held user accounts and state, and testing the whole thing in a simulator to make sure it actually behaved like an app and not just a set of screens. It was also the sprint that produced the single clearest result of the summer: every one of the six testers responded well to the friend’s feed, several of them comparing it, unprompted, to apps like Strava or Venmo that they already opened out of habit. One tester, who has ADHD and said he regularly loses track of where he left off in a show, called the memory-recall feature huge the first time he tried it.

It was also the sprint where real bugs showed up: a match score that displayed as five hundred percent, movies that vanished entirely from the habits screen because the underlying data only accounted for TV shows, a tracker that reset itself if you closed and reopened it from the home screen. None of these were catastrophic. All of them were the kind of thing you only find by watching someone actually use the thing you built, which is a large part of why the interviews mattered as much as the code did.

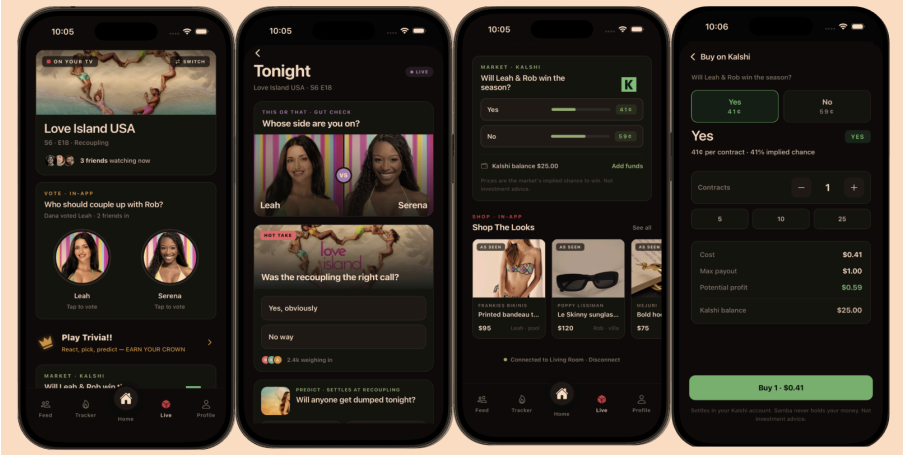


Sprint 3: Live, Shop, Trivia, and a Prediction Market

The third sprint was smaller by design, a mini sprint with three testers instead of six, and it tested the most experimental ideas: in-app voting on live reality shows, trivia tied to what was happening on screen, a shoppable carousel for products appearing in a show, and an integration with Kalshi, a real-money prediction market. This was the sprint that taught me the most about the difference between what I could build and what I should. Every one of those features worked, technically. The prediction market was the clearest miss of the entire summer regardless— not because it was broken, but because none of the three people who tried it wanted it there.

By the time I was heading into the fourth and final sprint, the pattern across the summer was already obvious. The building itself, once I had a working relationship with the tools, was rarely the bottleneck. The bottleneck was almost always deciding what was worth building at all, and then actually finding out, not guessing, whether it landed.

The fourth sprint picked up exactly where that left off. Rather than testing another flashy feature, it focused on the boring but necessary foundation the first three sprints had quietly been leaning on the whole time. This was a real onboarding flow and a privacy center that could actually explain, in plain language, what Samba does with a person's viewing data before asking them to hand any of it over.



IV. AI in My Development Workflow

I should say clearly what I mean when I say I "built" four prototypes, because it would be dishonest to imply I wrote all of that code myself in any traditional sense. I used Claude Code as my primary development tool the entire summer, and it is not an exaggeration to say that without it, this internship does not happen the way it did. What I actually did, most days, was closer to directing than typing: describing what a screen needed to do, feeding in real data structures from Databricks, reading through what got generated, running it in a simulator, and going back and forth until it worked and looked right. That is a real skill, and it took me most of June to get decent at it, but it is a different skill than the one I expected to need.

The clearest example of this is the Databricks work I mentioned earlier. Samba's whole hardware advantage is that its viewership data is real and enormous, and I wanted every prototype to reflect that instead of running on placeholder content, because testers can tell the difference immediately. Getting there meant learning to write and troubleshoot actual queries against a real data platform, with only one prior semester of basic Python behind me. I did that almost entirely by working through it with Claude: pasting in an error, asking what it meant, asking why a query returned the wrong shape of data, and slowly building up an understanding of Databricks that I did not have in June. I was not just copying answers. By the third sprint I could read a query and generally know what it was going to do before I ran it, which is not nothing, but I also know I would not have gotten there in ten weeks without an AI that could explain, in plain language, why my first few attempts were wrong.

I used AI in a couple of other ways beyond that core coding loop. For design decisions, such as how a rewards screen should actually look, how to visually separate two scores that testers kept confusing, I used Claude Design to generate several real, concrete options instead of

debating them in the abstract. That mattered because it let me A/B test actual choices with real testers rather than my own guess about what would read more clearly. Examples of this were things like the reward tier redesign, the way trivia cards displayed a user's friends, and the way a match percentage sat next to an aggregate score. Having two or three built options to put in front of someone during an interview, instead of one, made those sessions genuinely more useful.

For the backend plumbing, meaning user accounts, saved state, the parts of the app that needed to persist information between sessions, I worked with Firebase. Again here, mostly by having Claude walk me through setup and structure I had no prior experience with. And for debugging, which came up constantly given how fast each sprint moved, AI functioned less like a tool that fixed things for me and more like a translator. When the match score displayed 500 percent instead of a real percentage, or when a tracker reset itself after being closed and reopened, I did not understand the underlying code well enough to spot the bug by reading it. I understood it well enough, with Claude explaining what was happening step by step, to know what question to ask and what to check. That is a meaningfully different skill than debugging used to require, and I do not think it is a lesser one, but it is worth naming honestly.

AI showed up in one more place I have not mentioned yet, which is the writing itself. Every sprint ended with a synthesis document— a summary of what six interviews actually revealed, organized into themes, backed by direct quotes, and ending in a ranked list of what to build next. Turning six hours of raw conversation into something a product lead could read in ten minutes and actually trust is its own kind of work, and I used Claude here too, mostly to help organize scattered notes into consistent categories and to sanity-check that a pattern I thought I was seeing across testers actually held up when I laid the quotes next to each other. That is worth naming as its own category of AI in development, separate from writing code, because a lot of

what determined whether a sprint's findings actually got acted on was whether the write-up itself was clear enough to act on, and that had nothing to do with Databricks or Firebase at all.

I noticed I treated these two uses of AI differently without really deciding to. When Claude wrote a Databricks query, I ran it and let the result tell me whether it was right or wrong. When Claude helped me organize interview findings, there was no equivalent test; the only check was whether the summary still sounded like the people I had actually talked to, which meant I had to reread the raw notes myself before I trusted the summary at all. Coding mistakes tend to announce themselves. Synthesis mistakes are quieter, and I think that quietness is exactly why they are easier to miss.

I will add one more detail because I think it matters: I was doing all of this while also building a small personal app project on the side, mostly as a place to practice the same skills somewhere lower-stakes than Samba's actual data. Using Claude Code on my own project in parallel with using it at work turned out to reinforce both, a pattern I had not expected going in, and one that made me faster at the internship work specifically because I was not only ever encountering a new kind of problem for the first time during a testing week.

There is a specific kind of debugging that is easy to overlook, which is understanding code you did not write well enough that you can safely change it later. Because sprints were built directly on top of each other I was constantly opening a screen Claude and I had built together two or three weeks earlier and needed to modify it without breaking whatever was already working. That is a different task than writing something new from a blank prompt. It meant reading through generated code I had not personally typed, in a language I only partially knew, and asking Claude to walk me through what a given block was actually doing before I felt safe changing it. I got noticeably faster at this over the summer, not because I was memorizing

syntax, but because I was getting better at asking the right question when something looked unfamiliar. That distinction matters, because it means the specific skill I built this summer was not exactly coding in the way my computer science friends would recognize it. It was something more like technical literacy: the ability to look at a system, including one an AI wrote most of, and reason about whether a proposed change was safe, without necessarily being able to produce that system from nothing myself. I do not know yet whether that is a smaller skill than traditional programming or just a different one, but it was clearly enough to get the job done for ten weeks, and I suspect it will matter more, not less, as more of the actual typing keeps shifting onto tools like this one.

By the end of the summer, I had absolutely built real, working software. I had also outsourced a huge share of the actual writing of that software to a tool that did it faster, and frankly better, than I could have on my own. Both of those things are true at once, and figuring out what to make of that is most of what the rest of this paper is trying to do.

V. How AI Is Changing Software Development

My own summer is a small, specific data point, but it lines up with a much bigger shift that is visible across the industry right now, and it is worth stepping back from my own experience to describe it in more general terms.

The most measurable change is speed. A recent longitudinal study tracking developers with and without GitHub Copilot found consistent, meaningful gains in how quickly real engineering work got done (Stray et al., 2026), and McKinsey has reported similar results at a larger scale, estimating that generative AI tools can cut the time developers spend on certain coding tasks by roughly half (McKinsey & Company, 2023). I do not have a controlled study of my own summer, but I know that building a working, data-connected app prototype from scratch in roughly ten working days, which is what each of my sprints required, would not have been a realistic ask for someone with my technical background five years ago, AI or no AI. The two-week sprint cycle Samba built the whole internship around only makes sense in a world where a non-engineer can produce something functional enough to user-test in that window.

The second, related shift is accessibility, and this is the one I felt most personally. The programmer and AI researcher Andrej Karpathy coined the term “vibe coding” in early 2025 to describe a style of building software where you mostly describe what you want and let an AI model handle the implementation, to the point that you “forget that the code even exists” (Karpathy, 2025). That description is a little tongue-in-cheek, but it is a genuinely accurate label for a lot of what I did this summer, even though I had not heard the term before starting this paper. The bar for building real software has dropped in a way that is easy to underestimate if you did not grow up assuming it was closed off to you in the first place. I did assume that, as an International Relations and History major, and I was wrong.

The third shift is in what skill actually means, day to day. It is tempting to think AI just makes the same old skill, writing correct syntax, faster, but that is not quite what I experienced. The skill that mattered most for me was closer to specification: describing a problem precisely enough, and with enough of the right context, that the tool could act on it usefully. A vague request produced something generic; a request with the actual data shape, the actual constraint, and the actual failure mode I was seeing produced something I could use immediately. A 2024 analysis of generative AI's effect on software engineering work found a similar pattern industry-wide: the tasks getting automated fastest are implementation-heavy ones, while tasks requiring problem framing, requirements clarity, and evaluation increasingly fall to the human in the loop (Minkiewicz, 2024). That tracks with my summer almost exactly. The building got faster. The framing and evaluating did not get any faster or easier, and if anything I noticed my time shifting more and more toward those two things as the summer went on.

There is a more specific version of this shift worth naming on its own: prototyping itself has become disposable in a way it used to be too expensive to be. Traditionally, building a working prototype was still expensive enough that teams tried to be right the first time, or close to it, because throwing away two weeks of engineering work was a real cost. My summer ran on the opposite assumption. The third sprint's prediction-market feature is a good example. It took roughly the same amount of build time as anything else that sprint, it got tested honestly, it got a clear negative signal, and it got dropped, all inside two weeks, without anyone treating that as wasted effort. That only works if building the thing in the first place was cheap enough to treat as a real question rather than a bet. AI is what made it cheap enough, and I think that changes the psychology of building software just as much as it changes the mechanics of it.

AI also changes what learning a new tool looks like using it for real work, which is different from learning it in a classroom ahead of time. I did not take a Databricks course before touching Databricks. I learned it by breaking things inside a real sprint deadline and asking, in the moment, why the thing I just tried had not worked, which meant the learning and the doing happened at the same time instead of one before the other. That is a genuinely different model of picking up a skill than the one my college classes are built around, and I am not sure yet whether it produces shallower knowledge, faster useful knowledge, or, more likely, some of both at once.

None of this is a story about AI being uniformly trusted or loved, though, and I do not want to leave that impression. Stack Overflow's most recent developer survey found that even as AI tool adoption keeps climbing among professional developers, trust in the accuracy of what these tools produce has actually been falling, with a large share of developers reporting that AI-generated answers are almost right, but not quite, often enough to require real verification every time (Stack Overflow, 2025). That gap, using something constantly while trusting it conditionally, matched my own relationship with Claude almost exactly, and I think it is the more honest version of the "AI is changing software development" story than either the utopian or the doom-laden version usually gets credit for. It is not that AI removed the need for judgment. It is that judgment moved to a different point in the process, away from can I write this correctly and toward is this actually right, and how would I know.

Debugging is probably the clearest example of a task that changed in kind, not just in speed. The traditional model treats a bug as something you find by reading code carefully and reasoning from cause to effect. My summer worked more like the reverse: I would notice an effect first, a match score reading five hundred percent, a tracker resetting itself, and only afterward, with Claude's help, work backward toward a plausible cause. That is a real form of

debugging, and it has real problems fixed inside real deadlines, but it depends on trusting the explanation you are given about what went wrong, which is a different kind of trust than verifying it yourself line by line. That dependence is exactly why the accuracy question keeps coming up across every serious study of these tools right now, and it is why I do not think "AI makes debugging easier" is a complete sentence on its own. It makes debugging faster for people who could not have done it unassisted at all, which was true of me, and it introduces a new failure mode for exactly the same reason: the person doing the debugging may have no independent way to tell a right explanation from a plausible-sounding wrong one. I got lucky most of the time. I do not think luck is a strategy, and it is the main reason I take the next section's risks as seriously as I do.

That reframing is really the hinge of this whole paper, and it is what the next section is about directly: if a huge amount of the actual writing of code has shifted onto an AI tool, what is a developer, in the traditional sense, still responsible for?

VI. The Developer's Role in an AI World

If AI can generate a working recommendation engine, a rewards system, a live-voting feature, and a prediction-market integration inside a two-week window, and it can, because I watched it happen four times, then the obvious question is what a developer is actually for anymore. My internship gave me a partial answer, and it is less about any technical skill and more about five things that AI could not do for me, no matter how good the underlying model got.

The first is judgment about what to build in the first place. Nobody generated the idea to test recommendations and rewards before testing something more experimental like a prediction market. That sequencing, discovery before engagement before interactive features, was a deliberate strategic call made by people on my team who understood that you learn different things at different stages of a product's maturity. AI can build almost anything you describe to it. It cannot tell you which of the many things worth building is actually worth building first.

The second is verification, and this is the one my internship centered around most directly. Every prototype I built got tested with real people in structured, moderated interviews before anyone treated it as a real signal, and that process, the profile assessment, the hands-on testing, the closing discussion, existed specifically because a feature running correctly and a feature working for a human being are two completely different claims. AI could tell me my code ran. It could not tell me that six out of six people found the reward tier system incomprehensible. Only sitting across from those six people, and then noticing the pattern across their sessions, could do that.

The third is context that lives outside any one conversation with a tool. When a tester named Vidur told us that a real-money prediction market inside a TV app "doesn't make sense"

and was not “something he expects from this app at all,” that comment only became useful because I could connect it to what two other testers had said in different words, and to what I already knew about Samba’s brand as a data company rather than a betting company. An AI model reading that one transcript in isolation might have flagged it as negative feedback on a UI detail. Understanding it as a signal that the entire feature was a poor fit for the product, not just poorly designed, required a kind of accumulated context that no single tool call gave me.

The fourth is scope and architecture, deciding what belongs in a product at all, independent of whether it is technically buildable. The prediction market is the cleanest example from my summer: it was fully functional, reasonably well designed, and still the single worst-received feature we tested all summer. The constraint was never can we build this. AI made that question almost irrelevant. The constraint was should this exist here, and that is a judgment call about fit, trust, and brand that has nothing to do with implementation.

The fifth is communication, specifically the work of turning messy, human, sometimes contradictory feedback into something a team can actually act on. Every sprint ended with a ranked list: what to keep, what to kill, what to change, in that order, with reasoning attached. Writing that list well, in a way that someone who was not in the room could read and trust, was some of the hardest and most valuable work I did all summer, and it had almost nothing to do with code.

All five of these showed up at once during the reward-tier redesign in the first sprint, which is probably the clearest single example from my summer of what the job actually was. Deciding that rewards deserved a second look at all was judgment. Learning that six out of six people could not explain their own tier from watching the interviews was verification. Recognizing that the confusion was really about trust, since testers used words like “immediate

distrust" rather than just "unclear," was context. Deciding to scrap the point-to-tier structure entirely rather than patch the labeling was a scope call. And writing that decision up clearly enough that engineering treated it as a real recommendation rather than an intern's opinion was communication. AI built the original screen in an afternoon. None of the five things that actually determined what happened to that screen next came from the same place.

A second example makes the same point from a different angle. Whether shoppable content should check out inside the app or hand a person off to an outside retailer was a real, unresolved fork by the end of the summer, and it was unresolved specifically because it was not a technical question. Either version was buildable in an afternoon. The actual decision depended on things like how much friction Samba wanted between a viewer and a purchase, whether the company wanted to own that transaction at all, and how a rushed, accidental purchase during a live show would reflect on the trust built up everywhere else in the app. Nobody was going to get that answer from a model. It had to come from watching three different testers want three different things and then making a call about which of them the product could actually afford to prioritize.

Put together, those five things sketch out a version of "developer" that looks a lot less like person who writes code and a lot more like person who decides what code should exist, and who can tell, with evidence, whether it actually worked. That is not a smaller job. If anything, I found it to be a harder one.

VII. Limitations and Risks of AI-Assisted Development

None of what I have described should be read as an uncomplicated success story, and I want to spend this section on the parts that worried me, some of which I only recognized after the summer ended.

The most immediate risk is that AI-generated code can be subtly wrong in ways you cannot catch if you do not have the expertise to catch them. My prototypes shipped with a match score that briefly displayed as five hundred percent, genre metadata that mislabeled a comedy as a romance, and a deep-linking bug that showed content from a platform a tester did not even subscribe to. None of these were dramatic failures, but all of them made it into a testing round before anyone noticed, and I do not think that is a coincidence. I did not have the background to review the underlying code the way a trained engineer would have, so problems that a more experienced developer might have caught earlier, I caught only because a user happened to run into them live. That is a small-scale, low-stakes version of a much more serious documented risk: researchers studying AI-generated code have found that these tools can confidently invent things that do not exist at all, including software packages that sound real but are not, which attackers have already started exploiting by publishing malicious versions of those imagined packages (Spracklen et al., 2025). My bugs were embarrassing. That version of the same underlying problem is a real security hole.

The second risk is technical debt that builds up precisely because building is so fast. Moving through four sprints in ten weeks meant I rarely had time to go back and unify decisions made in earlier rounds, and it showed: trivia cards displayed a user's connections as "friends" on some screens and "fans" on others with no clear reason, a favorites list and a watchlist ended up functioning as the same feature under two different names because nobody had gone back to

differentiate them, and a tab literally named “Community” turned out to function as a personal profile page instead. None of these are hard problems. They are the kind of small inconsistency that accumulates specifically because AI removes the friction that used to force you to slow down and reconcile decisions before moving forward.

The third risk is building something because you can, rather than because you should, and the prediction-market feature is the clearest example from my own summer. It took no longer to build than any other feature that sprint. It was also the single worst-received idea we tested all summer, for reasons that had nothing to do with execution. It simply did not belong in a TV-viewership app, no matter how well it worked. Lowering the cost of building something does not lower the cost of it being the wrong thing to build, and I think that distinction is easy to lose when everything feels fast and possible.

The fourth risk is a genuinely open question I do not have a clear answer to: how much of Databricks and Python did I actually learn this summer, versus how much did I learn to describe problems well enough that I never had to? I can write a working query today. I am considerably less confident I could explain, from first principles, everything happening inside one. That is not a hypothetical concern. It is a documented shift, with a recent industry survey finding that even as professional developers rely on AI tools more than ever, their trust in the accuracy of what those tools produce has been declining, not rising (Stack Overflow, 2025). I lived that same tension without having a name for it: using Claude constantly, while never fully able to verify its output independently.

The last risk is one of scale that I could only partially see from where I sat. Everything I built was a prototype, tested in isolation, disconnected from Samba’s actual production codebase and the hardware-level infrastructure the whole company runs on. A ten-week internship

building disposable prototypes is a genuinely different task than maintaining software that has to survive inside a much larger, older, more tangled system, and I do not think anything about my summer taught me what that harder version of the job actually requires.

VIII. What My Internship Taught Me About the Future of Software

If you had asked me in May what a software engineering internship would look like, I would have described something close to what I had seen in movies: rows of monitors, long stretches of silent typing, code review as the emotional climax of the week. What actually surprised me was how little of my time, even in weeks I was actively building, was spent writing anything, and how much of it was spent watching someone else use a screen and trying to figure out why they hesitated before tapping a button.

That shifted how I think about the job itself. I came in assuming a software engineer was, by definition, someone who writes code, and that AI tools were something layered on top of that identity to make the writing faster. I left thinking the definition was closer to someone who decides what a product should do, builds evidence for whether that decision was right, and revises based on what they find, and that writing code, while still necessary, is no longer the part of that definition that requires the most specifically human judgment. AI absorbed a huge amount of the middle step for me this summer. It did not absorb the parts on either side of it.

The skills that mattered more than I expected, going in, were almost all about people rather than code: getting a stranger to actually tell you what confused them about a screen instead of politely saying it was fine, noticing that three different testers were describing the same underlying problem in three different vocabularies, and being willing to recommend killing a feature I had personally spent a week building because the evidence said to. None of those are skills I associated with “software development” before this internship. I am fairly confident now that they are the actual core of it, at least at the product layer where I was working.

The skill that mattered less than I expected was raw technical fluency, not because it stopped mattering entirely, but because the floor for it dropped so far that it stopped being the

bottleneck. I came in with one semester of basic Python and left having shipped four data-connected prototypes, and the gap between those two facts was not made up mostly by me getting dramatically better at programming in ten weeks. It was made up by a tool that let someone at my starting level participate in work that used to require years of training first. I do not think that makes the training worthless. I noticed real limits to what I could do without it, especially anywhere I needed to catch a subtle error myself. But it does mean the training is no longer the only door into doing the work.

If there is one thing I would tell a version of myself from this past May, it is that the scariest-sounding part of the internship description, software development, turned out to be the part I needed the least specialized preparation for. The part that actually determined whether I was good at the job all summer was the part nobody puts in the job title at all.

IX. Conclusion

I opened this paper with a question: if AI can write the code, what is a developer actually for? My internship at Samba TV is not a large enough sample to answer that question for the whole industry, but it gave me a specific, concrete version of an answer, and I think it holds up better than either of the two answers you hear most often.

It is not that AI is quietly replacing programmers, one prototype at a time, and it is not that AI is a minor convenience bolted onto a job that otherwise looks the way it always has. What I actually watched happen, over four sprints and ten weeks, was the value of the job sliding up a level. The work that used to be the hard, credentialed core of software development, writing a correct query, standing up a backend, tracking down a bug in a language you only half know, became fast enough and accessible enough that someone with my background could do a real version of it inside a two-week sprint. What did not get faster, and what ended up filling almost all of the time that freed up, was everything above that layer: deciding what was worth building, proving whether it actually worked for a real person, and being willing to say no to something that ran perfectly fine but still did not belong.

Yulia, one of my first testers back in June, told me she felt lied to by a recommendation engine that had a financial reason to push its own content at her. I did not think much of that comment in the moment beyond good insight, write it down. By the end of the summer, after watching testers pick apart four different prototypes for reasons that were almost never about whether the code worked, I understood it differently. She was not describing a technical failure. She was describing the exact kind of thing AI cannot verify on its own, whether something is actually trustworthy to the person using it, and that gap is precisely where I spent most of my summer working.

I do not know exactly what my next internship, or my first real job, will look like, or how much further this shift will go before I graduate. I do know that I walked into Samba TV assuming software development was a door closed to anyone without a computer science degree, and I walked out having built four real, working, data-backed products without one. That was only possible because AI absorbed the part of the job that used to gatekeep it. It is also, I would argue, exactly why the rest of the job, the part AI did not absorb, matters more than it used to, not less. Software is not being written by fewer people. It is being built by people who spend a lot less time typing and a lot more time deciding, testing, and judging whether what got built is actually any good. I spent my summer doing the second kind of work almost by accident, and I think that is where this is all heading, degree or no degree.





References

AlternativeTo. (2025, February). Trakt.tv has set stricter limits for free users and raised VIP subscription prices by 100%.https://alternativeto.net/news/2025/2/trakt-tv-has-set-stricter-limits-for-free-users-and-raised-vip-subscription-prices-by-100-/

Karpathy, A. (2025, February 2). There's a new kind of coding I call "vibe coding" [Post]. X. https://x.com/karpathy/status/1886192184808149383

McKinsey & Company. (2023). Unleashing developer productivity with generative AI. https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/unleashing-developer-productivity-with-generative-ai

Minkiewicz, A. (2024). The impact of generative AI on software engineering activities. U.S. Department of Homeland Security. https://www.dhs.gov/sites/default/files/2025-01/2024_1219_impact_of_genai_on_software_engineering_activities_minkiewicz.pdf

Spracklen, J., Wijewickrama, R., Sakib, A. H. M. N., Maiti, A., Viswanath, B., & Jadliwala, M. (2025). We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. USENIX Security Symposium; login: Online. https://www.usenix.org/publications/loginonline/we-have-package-you-comprehensive-analysis-package-hallucinations-code

Stack Overflow. (2025). 2025 Stack Overflow Developer Survey: AI. https://survey.stackoverflow.co/2025/ai

Stray, V., Brandtzæg, E. G., Wivestad, V. T., Barbala, A., & Moe, N. B. (2026). Developer productivity with and without GitHub Copilot: A longitudinal mixed-methods case study. arXiv. https://arxiv.org/abs/2509.20353